NELC Compliance

How PhysioGuidance meets Saudi NELC accreditation for continuing physiotherapy education — xAPI 1.0.3 statement stream to the NELC LRS, SCFHS export, CPD activity register, learner-journey evidence, and the full evidence matrix.

PREPARED FOR

PhysioGuidance Client Team

DATE

2026-08-17

LRS ENVIRONMENT

Runtime toggle (staging or production)

PHYSIOGUIDANCE

Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

1. Executive summary
2. NELC integration architecture
3. xAPI statement stream (1.0.3)
4. LRS environment toggle (staging / production)
5. LRS resilience — persist, worker, cron sweep
6. Learner journey — actions that emit xAPI statements
7. Course compliance scorecard (5 categories)
8. Session compliance rules
9. Certificate issuance, verification, and revocation
10. NELC audit log (immutable event trail)
11. Admin NELC integration endpoints
12. Course-level evidence bundle (ZIP)
13. Evidence matrix + screenshot bundle
14. Component status matrix
15. Related deliverables
16. Where to look next

Deployed commit: `d396b040`.

---

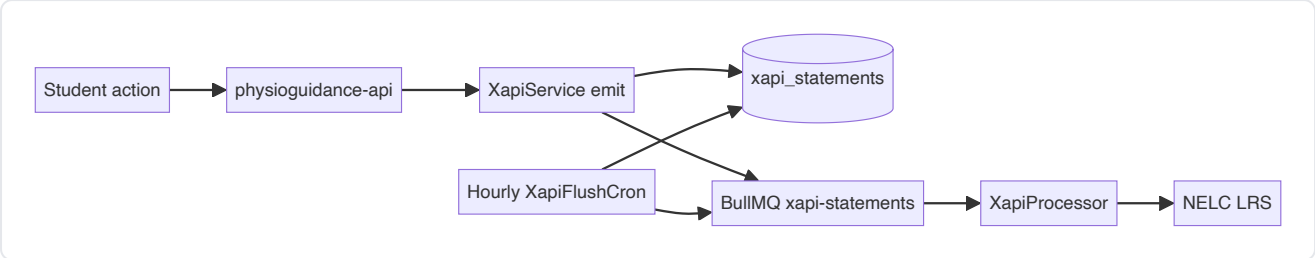
1. Executive summary

Every learner event on PhysioGuidance emits an xAPI 1.0.3 statement to NELC's LRS via a persistent, retry-safe pipeline. Statements are first written to Postgres, then dispatched by a BullMQ worker that calls the LRS with in-client exponential backoff, then swept hourly by a cron job that picks up any pending or failed rows. Certificates carry a public verification URL and a QR code linking to it. Course-level compliance is scored across five categories that map to NELC clauses, and every admin action (enrollment change, certificate revoke, session finalize, Zoom sync) writes to an immutable NELC audit log.

The full per-control evidence matrix is in nelc-evidence-matrix.csv. Screenshot bundle in `nelc-screenshots/` folder. Overview in NELC-EVIDENCE-README.md.

---

2. NELC integration architecture



Two-stage durability: every statement is persisted to Postgres BEFORE dispatch. If Redis or the LRS is unreachable, the row stays `pending`. The hourly cron re-enqueues anything stuck in `pending > 10 min` or `failed > 30 min`. No learner request is blocked by LRS latency.

Source modules:

- `backends/src/modules/xapi/` — statement builders, LRS client, processor, admin controller
- `backends/src/modules/nelc-compliance/` — per-course + per-session scoring, evidence bundle
- `backends/src/modules/nelc-audit/` — immutable NELC audit log query + CSV export
- `backends/src/config/nelc.config.ts` — LRS endpoints, credentials, timeouts, retries

3. xAPI statement stream (1.0.3)

Ten NELC-recognised verbs are emitted, each with a purpose-built statement builder that follows the NELC PHP SDK reference shape:

Verb IRI	High-level emitter	Trigger
`http://adlnet.gov/expapi/verbs/registered`	`emitCourseEnrollment`	Enrollment row created
`http://adlnet.gov/expapi/verbs/initialized`	`emitCourseInitialized`	First open of course content after enrollment
`https://w3id.org/xapi/acrossx/verbs/watched`	`emitVideoWatched`	Video material watched (partial or complete)
`http://adlnet.gov/expapi/verbs/completed`	`emitLessonCompleted` / `emitCourseCompleted`	Module or course completion
`http://adlnet.gov/expapi/verbs/progressed`	`emitCourseProgressed`	Monotonic course-progress percentage update
`http://adlnet.gov/expapi/verbs/attempted`	`emitAssessmentSubmitted`	Assessment submitted, with score + attempt number
`http://id.tincanapi.com/verb/rated`	`emitCourseRated`	Course rating submitted (raw/min/max 0-5)
`http://id.tincanapi.com/verb/earned`	`emitCertificateIssued`	Certificate approved, PDF URL attached
`http://adlnet.gov/expapi/verbs/logged-in`	`emitLogin`	User login (Amna checklist step 1)
`http://adlnet.gov/expapi/verbs/attended`	`emitWebinarAttended`	Live webinar / virtual session attendance

Statement shape — every emitted statement follows xAPI 1.0.3 spec and adds NELC's custom extensions under `https://nelc.gov.sa/extensions/*`:

- `actor` — learner identity (mailto:, name, and NELC extensions for national ID, mobile, full name, nationality, DOB)
- `verb` — one of the ten IRIs above, with `en-US` display map
- `object` — course, module, video, assessment, or certificate activity (canonical URL as `id`)
- `result` — score `raw/min/max` for assessments, duration for attendance, completion + success flags
- `context` — `platform` identifier, `instructor` agent, `contextActivities.parent` (always the course), `language` in BCP 47 (`ar-SA` or `en-US`), plus NELC extensions (`platform`, `lms_url`, `program_url`, `duration`, `learner_*`)
- `id` — reused across retries so the LRS deduplicates on the statement UUID (checklist SEQ003)
- `timestamp` — ISO 8601 UTC

Persistence table: `xapi_statements` (columns for verb, actor, course, enrollment, object activity ID + type, status, plus the full JSON payload). Per-attempt transmission history lives in `xapi_transmissions`.

Version header: `X-Experience-API-Version: 1.0.3` on every LRS POST.

Duplicate guard: webinar `attended` statements can be triggered from three paths (Zoom webhook, Zoom sync cron, manual mark). `XapiService.emitWebinarAttended` checks for an existing non-failed `attended` statement for the same (learner, session) pair before emitting.

Provider triggers: `emitProviderJoined` / `emitProviderLeft` send the one-time administrative statements NELC requires when a platform onboards or offboards from the LRS. Triggered manually from the admin panel.

---

4. LRS environment toggle (staging / production)

Two LRS environments coexist. The active one is resolved fresh on every LRS POST through `NelcModeService`:

1. `system_settings.nelc.mode` (admin UI toggle) — persisted in the database
2. `NELC_LRS_MODE` env var (default)
3. `staging` (fail-safe)

Toggle endpoint: `POST /admin/integrations/nelc/mode` — restricted to **SUPER_ADMIN**. Refuses to flip if the target mode's password isn't loaded (won't half-cut over).

Cache: 30-second in-process cache; invalidated the moment the admin saves the `nelc` settings category, so the next LRS POST picks the new endpoint within milliseconds.

Endpoints & credentials (from `nelc.config.ts`):

Field	Staging	Production
Endpoint env	`NELC_LRS_ENDPOINT_STAGING`	`NELC_LRS_ENDPOINT_PROD`
Password env	`NELC_LRS_PASSWORD`	`NELC_LRS_PASSWORD_PROD`
Username env	`NELC_LRS_USERNAME` (shared)	`NELC_LRS_USERNAME` (shared)
Auth scheme	HTTP Basic (base64 of `username:password`)	HTTP Basic (base64 of `username:password`)

Every mode change writes `system_settings.updated_by` / `updated_at` for the audit trail.

---

5. LRS resilience — persist, worker, cron sweep

Three-layer resilience:

5.1 In-request persist

`XapiService.emit()` writes the statement to Postgres synchronously (fast local write), patches the statement UUID into the payload's `id` field so retries are idempotent, then enqueues. The learner never waits on LRS latency.

5.2 BullMQ worker with in-client retries

`XapiProcessor` (concurrency 4) drains the `xapi-statements` queue. Each queue job runs `attempts: 1`; the retry logic lives inside `LrsClient.postStatement`:

- 2xx → mark `sent`, stamp `sent_at`
- 4xx → mark `failed` immediately (client bug, no retry)
- 5xx / timeout / network error → exponential backoff (500ms, 1s, 2s, …) up to `NELC_LRS_MAX_ATTEMPTS` (default 3), then `failed`
- Timeout per attempt: `NELC_LRS_TIMEOUT_MS` (default 15 s), using `AbortController`

Every attempt writes a row to `xapi_transmissions` with `http_status`, `response_body`, `error_text`, and `attempt_no`.

5.3 Hourly sweep cron

`XapiFlushCron` runs `@Cron(EVERY_HOUR)`, guarded by a distributed lock (`lock:cron:xapi.flush`, 8 min TTL) so multi-instance deploys never double-enqueue. Each tick re-scans `xapi_statements` for:

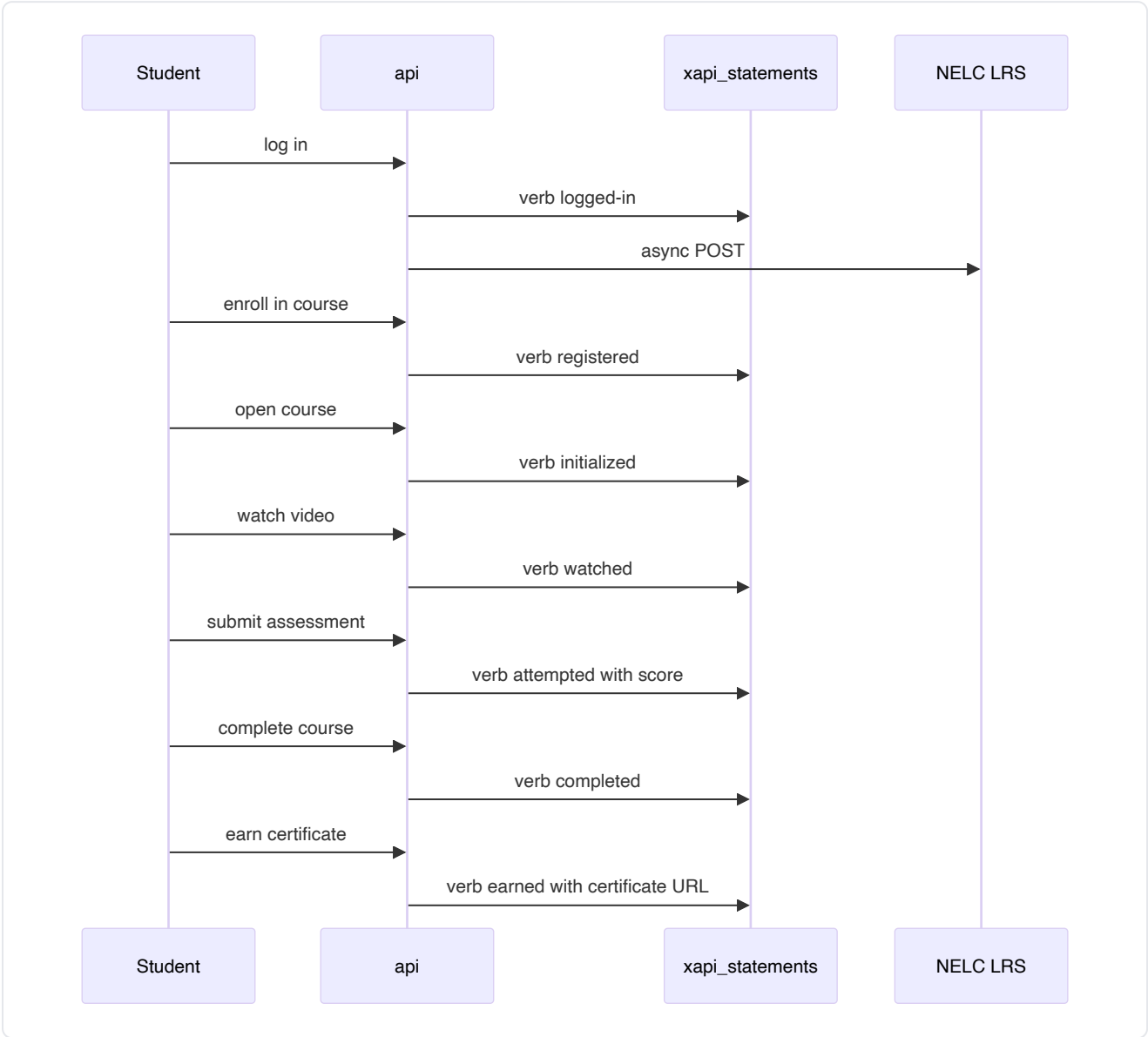
- `pending` rows older than 10 minutes (queue lost the job, e.g. Redis restart)
- `failed` rows older than 30 minutes (automatic retry cool-down)

Capped at 500 statements per tick so a backlog can't monopolise the queue. Externalisable via the `EXTERNALIZED_CRONS` env flag when the paired Cloud Run Job takes over.

Audit trail: every dispatch attempt is visible in the admin log with HTTP status, response body, and error text. Admins can re-enqueue individual statements from Admin → Integrations → NELC → Retry.

---

6. Learner journey — actions that emit xAPI statements



Screenshots documenting each stage of the learner journey with corresponding xAPI statement dispatch are in the `nelc-screenshots/` folder — see `user-journey-nelc.png` and `nelc-user-journey.png`.

7. Course compliance scorecard (5 categories)

`NelcComplianceService.getCourseComplianceScorecard()` returns a five-category scorecard per NELC spec, with per-category status (`passed` / `review` / `failed`) and an overall percentage score. Categories adapt to the course's delivery type — self-paced skips the virtual-classes category, onsite skips Zoom configuration, and so on.

Category	Rules checked
Course Structure	`modules_count` (min 3), `modules_with_materials` , `bilingual_content` (EN + AR titles), `bilingual_descriptions` , `cme_hours_configured`
Accessibility	`wcag_status` (all materials WCAG compliant), `video_captions` (all videos captioned), `video_transcripts` (all videos transcribed)
Virtual Classes	`sessions_exist` , `zoom_configuration` (all Zoom/hybrid sessions have meeting link), `attendance_tracking` (completed sessions have attendance rows), `ta_allocation` (thresholds below)
Assessment Quality	`assessments_exist` , `blueprint_coverage` (percentage of modules with an assessment), `graded_submissions`
Instructor Qualifications	`instructor_role` (has active INSTRUCTOR role), `credentials_uploaded` (resume/CV on the role request)

Overall score: `passed_categories / total_categories * 100` . Overall status is `passed` at 100, `review` at 50–99, `failed` below 50.

TA-allocation thresholds (session-level check, `NelcComplianceService`):

- ≤ 25 enrolled → no TA required
- 26 – 35 enrolled → 1 TA required
- ≥ 36 enrolled → 2 TAs required

Enrollment cap (from `courses/nelc-config.ts`): default 2000 learners for synchronous courses (live / Zoom / hybrid), adjustable per course by admin; unlimited for self-paced.

Endpoints:

- `GET /nelc-compliance/summary` — dashboard tile counts (passed / review / failed)
- `GET /nelc-compliance/course/:id` — per-course rollup across sessions
- `GET /nelc-compliance/course/:id/scorecard` — 5-category scorecard with per-rule detail
- `GET /nelc-compliance/all-courses` — all-courses matrix for the CM dashboard page
- `GET /nelc-compliance/export.csv` — one row per course with score + per-category breakdown (BOM-prefixed UTF-8 so Excel renders Arabic titles)

- `GET /nelc-compliance/virtual-classes` — per-session compliance for Zoom / hybrid sessions

---

8. Session compliance rules

`NelcComplianceService.validateSessionCompliance(sessionId)` runs five checks:

1. `enrollment_limit` — active enrolments vs NELC cap (2000 default); flags `review` at 90 % of cap, `failed` above
2. `assistant_allocation` — required TAs from thresholds above vs assigned `session_assistants`
3. `zoom_configuration` — Zoom / hybrid sessions must have `meetingId` or `zoomLink`
4. `scheduling` — session has `scheduledAt` and `durationMinutes >= 15`
5. `attendance_tracking` — completed sessions must have at least one row in the `attendance` table

Overall session status rolls up: any `failed` → `failed`, any `review` → `review`, else `passed`.

Webinar CME rule (≥ 75 % attendance): the platform enforces a 75 % attendance floor before a learner is credited for a webinar's CME hours. The threshold is applied by:

- `reports.service.ts` — flags any student below 75 % on the CME-award report
- `support.service.ts` — the `ATTENDANCE_THRESHOLD = 75` constant is used when reconciling attendance-driven eligibility

---

9. Certificate issuance, verification, and revocation

9.1 Issuance

- Certificate PDF is generated in-process by `certificates.service.ts` using PDFKit + `qrcode`, embedding Cairo (Arabic) and Amiri fonts for bilingual output.
- Student name, course title, CME hours (from `courses.cme_hours`), issue date, SCFHS accreditation number (`courses.accreditation_number`), and short certificate code (`PG-<16 hex>`) are printed on the PDF.
- QR code links to `${FRONTEND_URL}/en/verify/{requestId}` — the public verification page.
- `emitCertificateIssued` fires one `earned` xAPI statement per issuance, with the certificate download URL in `context.extensions[jws-certificate-location]` (NELC official sample #11).

9.2 Public verification

- **Endpoint:** `GET /certificates/verify/:id` — no authentication required.

- Accepts either a full UUID or the printed short code (`PG-XXXXXXXX…` , 8–32 hex chars). Legacy 8-char codes remain verifiable; new codes are 16 hex chars.
- Returns: `verified` boolean, status (`approved` / `revoked`), student name, course name (EN + AR), SCFHS accreditation number, CME total hours, issued date, and the printable certificate ID.

9.3 Revocation

- **Endpoint:** `POST /certificates/:id/revoke` — MANAGER / ADMIN only, requires a reason.
- Flips `status → revoked` , clears the PDF URL, and stamps `revokedBy` , `revokedAt` , `revocationReason` .
- Writes an immutable NELC audit event: `event_type: 'certificate_revoked'` , `entity_type: 'certificate_request'` , with `courseId` , `studentId` , `reason` in metadata.
- Sends the student an email notification with the revocation reason.

---

10. NELC audit log (immutable event trail)

Table: `nelc_audit_logs` . Insert-only by design — the service exposes `log()` (insert), `query()` (select), `getSummary()` (aggregate by event type), and `exportCsv()` (up to 10 000 rows). No update, no delete.

Schema: `id` , `event_type` , `entity_type` , `entity_id` , `user_id` , `metadata` (JSONB), `ip_address` , `created_at` . Indexed on `event_type` , `(entity_type, entity_id)` , `user_id` , and `created_at` .

Event catalogue (`NELC_EVENT_TYPE` constant, over 30 distinct types):

Category	Event types
Sessions	`session_started`, `session_ended`, `session_cancelled`, `session_finalized`
Attendance	`attendance_marked_manual`, `attendance_auto_synced`, `attendance_zoom_join`, `attendance_zoom_leave`, `attendance_unverified_confirmed`
Enrollment	`enrollment_created`, `enrollment_dropped`, `enrollment_limit_enforced`
TA allocation	`assistant_allocated`, `assistant_deallocated`
Compliance	`compliance_check_passed`, `compliance_check_failed`
Zoom	`zoom_meeting_created`, `zoom_sync_completed`, `zoom_join_link_generated`
Certificates	`certificate_revoked`
Course lifecycle	`course_submitted_for_review`, `course_changes_requested`, `course_activated`, `course_deactivated`, `course_cancelled`, `course_rescheduled`, `course_ended_auto`, `course_ended_manual`, `course_completed_manual`, `course_deleted_from_ended`, `course_deleted_from_archive`

Endpoints:

- `GET /nelc-audit` — paginated, filterable query (ADMIN / SUPER_ADMIN)
- `GET /nelc-audit/summary` — aggregate counts by event type for a date range (ADMIN / SUPER_ADMIN)
- `GET /nelc-audit/export` — CSV download of filtered rows (MANAGER / ADMIN / SUPER_ADMIN)

---

11. Admin NELC integration endpoints

All under `/admin/integrations/nelc`, MANAGER / ADMIN / SUPER_ADMIN only unless noted.

Endpoint	Purpose
`GET /mode`	Current active mode, source (`db` / `env` / `default`), both endpoints, credential availability
`POST /mode`	Switch active mode (SUPER_ADMIN only) — refuses if target password missing
`GET /config`	Sanitised config view — endpoint, masked password, platform identity, xAPI version, timeouts, max attempts
`GET /stats`	Last-24h totals (sent / failed / pending), all-time pending + failed, last successful sync timestamp
`GET /log`	Paginated statement log with latest transmission HTTP status, error, and attempt count
`GET /log/:id`	Full JSON payload + all transmission attempts for one statement
`POST /retry/:id`	Re-enqueue a failed statement — flips `failed → pending` immediately for UI feedback
`POST /test-connection`	Probe LRS reachability with a self-contained `logged-in` statement (no DB row written)
`POST /trigger/joined`	Send NELC "LMS Provider Joined" one-time onboarding statement
`POST /trigger/left`	Send NELC "LMS Provider Left" offboarding counterpart

12. Course-level evidence bundle (ZIP)

`GET /nelc-compliance/course/:id/evidence` (MANAGER / ADMIN / SUPER_ADMIN) returns a ZIP archive containing everything a NELC auditor needs to reconstruct the course's compliance posture:

- `cover-summary.txt` — human-readable summary of contents + status
- `compliance-checks.json` — every session check with its NELC clause reference (`NELC-VC-*` , `NELC-CS-*` , `NELC-AC-*` , `NELC-AQ-*` , `NELC-PL-*` , `NELC-DP-*` , `NELC-IQ-*`)
- `sessions.csv` — every session with type, schedule, duration, Zoom link, meeting ID
- `attendance.csv` — full attendance log with join/leave timestamps + minutes attended
- `materials-audit.csv` — all materials with WCAG status, version, reviewer, review note
- `assessment-results.csv` — every submission with score, total marks, percentage
- `instructor-credentials.json` — role request status, profile data, resume files
- `student-feedback.csv` — course ratings with summary (count + average)
- `accessibility-report.json` — aggregate WCAG counts (pass / fail / pending) + video totals

- `policies.json` — instructor policy configuration (enrollment limit, content review, cancellation notice, NELC enforcement)

---

13. Evidence matrix + screenshot bundle

Full per-control matrix mapping each applicable NELC requirement to:

- Requirement / Control description
- Category (xAPI, LRS, Certificate, SCFHS, etc.)
- PhysioGuidance implementation summary
- System feature name
- Code module / file reference
- Evidence location (admin dashboard URL + API endpoint)
- Reference document + page number in prior handover PDFs
- API / DB / audit-trail reference
- Validation Status (Implemented / Not Applicable / Requires Client/Legal Action / Outstanding Development)

Delivered as nelc-evidence-matrix.csv.

Screenshot bundle in `nelc-screenshots/`:

File	Shows
`alerts-nelc-audit.png`	NELC-related alerts in admin dashboard
`audit-logs-evidence.png`	Audit log filtered to NELC actions
`audit-logs.png`	General audit log view
`certificates-verification.png`	Public verification page for an issued cert
`lrs-connection.png`	LRS connection status in admin
`nelc-user-journey.png`	Student journey through NELC-tracked events
`public-nelc.png`	Public-facing NELC compliance badge / info
`user-journey-nelc.png`	Alternate view of learner journey

Full index in NELC-EVIDENCE-README.md.

---

14. Component status matrix

Using Amna's four-category vocabulary.

Control	Status	Notes
xAPI 1.0.3 statement emission for all ten NELC verbs	Implemented	`XapiService` with dedicated high-level emitters per event
Persist-first durability + BullMQ dispatch + hourly cron sweep	Implemented	`xapi_statements` + `XapiProcessor` + `XapiFlushCron` (`EVERY_HOUR`)
In-client retry with exponential backoff on 5xx / timeout	Implemented	`LrsClient.postStatement` — 4xx short-circuits, 5xx retries up to `NELC_LRS_MAX_ATTEMPTS`
SUPER_ADMIN-only LRS environment toggle (staging / production)	Implemented	`POST /admin/integrations/nelc/mode` via `NelcModeService` (30 s cache)
Distributed cron lock for multi-instance safety	Implemented	`DistributedLockService.withLock('lock:cron:xapi.flush')`
Duplicate-statement guard for webinar attendance	Implemented	`emitWebinarAttended` skips if a non-failed `attended` row exists
LMS Provider Joined / Left administrative triggers	Implemented	`POST /admin/integrations/nelc/trigger/joined` and `/trigger/left`
5-category course compliance scorecard with delivery-type awareness	Implemented	`NelcComplianceService.getCourseComplianceScorecard()`
Per-session compliance rules (enrollment, TA, Zoom, schedule, attendance)	Implemented	`validateSessionCompliance()` with five checks

Control	Status	Notes
Course-level evidence ZIP bundle with NELC clause mapping	Implemented	`GET /nelc-compliance/course/:id/evidence`
All-courses compliance CSV export	Implemented	`GET /nelc-compliance/export.csv` (UTF-8 BOM for Arabic)
Certificate PDF with CME hours, SCFHS accreditation number, QR verification	Implemented	`certificates.service.ts` (PDFKit + qrcode)
Public certificate verification endpoint	Implemented	`GET /certificates/verify/:id` — accepts UUID or `PG-XXXX` short code
Certificate revocation with immutable NELC audit event	Implemented	`revoke()` writes `certificate_revoked` to `nelc_audit_logs`
Immutable NELC audit log with 30+ event types	Implemented	`nelc_audit_logs` (insert-only) + query + summary + CSV export
Webinar CME rule — ≥ 75 % attendance	Implemented	`ATTENDANCE_THRESHOLD = 75` in reports + support services
Evidence matrix + screenshot bundle	Implemented	nelc-evidence-matrix.csv + `nelc-screenshots/`

15. Related deliverables

- nelc-evidence-matrix.csv — per-control status + evidence pointers
- NELC-EVIDENCE-README.md — how to use the screenshot + evidence bundle

- `nelc-screenshots/` — UI screenshots showing NELC evidence
- pdpl-compliance.md — PDPL context (data subject rights, retention)
- security-architecture.md — how learner data is protected

---

16. Where to look next

- **The learner journey UI itself** → student-guide.md
- **How certificate PDFs render** → `backends/src/modules/certificates/certificates.service.ts`
- **LRS retry endpoint (manual re-dispatch)** → `POST /admin/integrations/nelc/retry/:id`
- **LRS environment toggle** → `POST /admin/integrations/nelc/mode` (SUPER_ADMIN)
- **NELC / xAPI operational monitoring** → monitoring-alerting.md